

毕业设计答辩讲稿

2026-05-25

毕业设计答辩讲稿参照

本文档对应当前 ppt/finalterm/2022112879-郑彦文-毕业设计答辩.pptx，共 22 页：第 1-16 页为正式汇报，第 17-21 页为备份页，第 22 页为结束页。口头汇报建议控制在 8-10 分钟，问答约 5 分钟。

使用时不要逐字背诵。每页抓住“页面任务、核心句、补充解释”三件事即可。答辩主线保持为：为什么需要这个系统 -> 我设计了什么架构 -> 我提出了什么 workflow 规划机制 -> 我如何实现 -> 我如何验证它有效。

一、整体讲法

页码	章节	建议时间	这一段要讲清楚的事
1-3	开场、目录、问题链	50 秒	本文是软件工程系统实现题，围绕 workflow 控制层展开
4-5	背景与边界	1 分钟	蛋白质设计工具链复杂、昂贵、会失败，因此需要可恢复可审计的控制层
6-8	架构设计	2 分钟	五层架构、多 Agent 职责边界、FSM/HITL 控制

页码	章节	建议时间	这一段要讲清楚的事
9-11	CEBRA-WP	2 分钟	算法解决运行时恢复决策，先硬过滤，再用状态重排合法候选
12-13	工程实现	1 分 30 秒	代码模块、WorkflowContext、StepRunner、ToolAdapter、日志与快照
14-16	验证与总结	1 分 50 秒	13 个系统测试、84-run 消融实验、贡献与结论边界
17-21	备份页	问答使用	失败样本、测试证据、界面接口、后续工作

答辩中需要反复守住三个边界：

- 本工作聚焦工作流控制层，不训练新的蛋白质生成模型。
- 本工作没有宣称湿实验验证，生物学有效性留待后续实验。
- CEBRA-WP 的主要结论是机制可执行、可观测、可恢复、可审计，并能减少部分无效高代价调用；不要把它说成功率显著提升。

二、逐页讲稿

第 1 页：封面

屏幕内容：题目“基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发”，答辩人郑彦文，软件工程，指导教师陈源龙，答辩日期 2026 年 5 月 26 日。

建议口播：

各位老师好，我是软件工程专业郑彦文。我的毕业设计题目是《基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发》。这项工作关注的重点是蛋白质设计工具链的工作流控制：如何把已有工具、Agent 规划、人工确认、失败恢复和证据审计组织成一个可运行的工程系统。

切页句：下面我先用目录说明整个汇报结构。

第 2 页：目录

屏幕内容：五个部分，分别是研究背景与问题定义、系统架构与协作边界、 workflow 规划机制、工程实现与测试验证、结论与备查材料。

建议口播：

本次汇报分五部分。第一部分说明问题背景和研究边界；第二部分介绍系统架构以及多 Agent 的职责边界；第三部分讲核心 workflow 规划机制 CEBRA-WP；第四部分说明系统如何落到 API、Workflow、ToolAdapter 和 Web 工作台；最后用测试、消融实验和备份材料说明验证结果与工作边界。

补充说明：

这页可以快速带过，重点句是：“本次答辩主要看工程系统本身：架构是否清楚、机制是否可解释、实现是否可信、验证是否充分。”

切页句：先看这套 PPT 围绕哪几个问题展开。

第 3 页：汇报问题链

屏幕内容：五个问题：为什么需要系统、设计什么架构、提出什么机制、如何工程落地、如何验证有效。

建议口播：

这页是整场汇报的问题链。我的讲述从工程约束出发：蛋白质设计 workflow 往往涉及多种工具，高代价工具不能盲目调用，流程中还可能出现参数错误、I/O 缺失或质量门禁失败。针对这些问题，本文先给出五层架构和多 Agent 协作边界，再介绍 CEBRA-WP workflow 规划与恢复控制机制，最后说明实现模块和验证证据。

可强调的一句话：

我希望老师记住的是：本文的评价对象不是某个蛋白生成模型效果，核心工作是实现一个面向蛋白质设计工具链的可规划、可执行、可恢复、可审计的控制系统。

切页句：具体来说，为什么这类系统有必要做，先从 workflow 复杂性看起。

第 4 页：蛋白质设计工作流的工程复杂性

屏幕内容：四个工程痛点：接口异构、高代价调用、失败可恢复、关键决策需审查；底部说明研究范围是已有蛋白质设计工具的工作流控制层。

建议口播：

蛋白质设计并不是单次模型调用就能完成。一个典型流程可能包括目标描述、候选序列生成、结构预测、质量筛选、打分和结果汇总。工程上会遇到四类问题：第一，序列、结构、评分工具的输入输出格式差异明显；第二，结构预测和重型打分属于高代价调用，证据不足时不适合直接执行；第三，参数错误、I/O 缺失、质量门禁失败都可能发生，但这些失败并不一定意味着任务必须结束；第四，高风险和高成本决策需要人工确认。

补充解释：

这里的关键在于工具之间缺少一个可靠的工作流控制层。已有工具有单点能力，本文要做的是把这些能力组织成可控流程。

切页句：因此，第一个需要明确的问题是本文到底解决什么、不解决什么。

第 5 页：本文解决的是工作流控制问题

屏幕内容：研究边界和四项工作：系统架构、核心机制、工程实现、验证实验。

建议口播：

本文的研究边界比较明确：不训练新的蛋白质生成模型，不宣称湿实验验证，也不替代 AlphaFold、ESMFold 等底层工具。本文解决的是工作流控制问题。我的主要工作包括四部分：设计五层架构和多 Agent 协作边界；提出 CEBRA-WP 规划与恢复动作选择机制；实现 API、Workflow、ToolAdapter 和 Web 工作台；最后用 13 个系统测试用例和 84-run 消融实验做验证。

可强调的一句话：

本人工作量集中在把任务、工具、状态、人工决策、恢复和证据链系统化打通。

切页句：明确边界之后，先看系统总体架构。

第 6 页：系统五层架构

屏幕内容：输入交互层、智能规划层、 workflow 执行层、安全与汇总层、资源层；底部架构原则：规划不执行工具，执行不越过 FSM，安全不改计划，汇总不重新计算。

建议口播：

系统采用五层架构。最上层是输入交互层，承接 Web、CLI、API 和人工决策。第二层是智能规划层，包括 PlannerAgent、ProteinToolKG 和 CEBRA-WP，用于生成候选计划和恢复候选。第三层是 workflow 执行层，包括 ExecutorAgent、PlanRunner、StepRunner，负责按已确认计划执行。第四层是安全与汇总层，SafetyAgent 负责风险信号，SummarizerAgent 负责结果汇总。底层资源层包含 ToolAdapter、工具知识图谱、日志、快照和产物。

补充解释：

这套架构的关键是职责隔离。Planner 只产出候选，不直接调用工具；Executor 只执行已确认步骤，不越过状态机；Safety 输出 ok、warn、block 风险信号；Summarizer 只汇总已有结果。这样可以把大模型规划的不确定性限制在结构化候选和可审计决策之内。

切页句：架构分层之后，还需要说明多个 Agent 如何协作而不越权。

第 7 页：多 Agent 协作如何避免越权

屏幕内容：PlannerAgent、ExecutorAgent、SafetyAgent、SummarizerAgent 的职责边界，以及 ProteinDesignTask、PlanCandidate、StepResult、PendingAction、Decision、TaskSnapshot 等核心契约。

建议口播：

这一页说明 Agent 间的协作边界。PlannerAgent 负责生成 Plan、Patch、Replan 候选，但不执行工具、不自行确认计划。ExecutorAgent 只执行已确认的 PlanStep，并在 WAITING 状态下停止继续调用工具。SafetyAgent 输出 ok、warn、block 风险信号，但不直接修改计划或终止任务。SummarizerAgent 汇总 DesignResult 和报告，不把未验证推断写成结论。

补充解释：

这些边界依赖结构化对象来约束。ProteinDesignTask 表示任务输入，PlanCandidate 表示候选，StepResult 表示步骤结果，PendingAction 表示待人工确认事项，Decision 表示人工或自动决策，

TaskSnapshot 用于恢复。系统可信性来自这些对象之间的约束关系，而不是让某个 Agent 自由决定所有事情。

时间紧时压缩讲法：

这页可以只讲一句：“多 Agent 协作主要靠结构化契约、状态机和事件日志约束，提示词只承担辅助说明。”

切页句：接下来是控制人工决策和恢复流程的状态机。

第 8 页：用状态机约束人在环决策

屏幕内容：WAITING_PLAN_CONFIRM、WAITING_PATCH_CONFIRM、WAITING_REPLAN_CONFIRM 三类等待态，底部说明 HITL 由 PendingAction、Decision、EventLog、Snapshot 共同约束。

建议口播：

系统用 FSM 管理任务生命周期，其中 HITL 对应几个明确的等待态。WAITING_PLAN_CONFIRM 用于初始高成本或高风险计划确认；WAITING_PATCH_CONFIRM 用于局部失败后的修补确认；WAITING_REPLAN_CONFIRM 用于整体风险、结构性失败或 stop 候选确认。进入等待态后，Executor 不会继续调用工具，前端只提供决策入口。用户提交 Decision 后，系统再根据状态机规则继续执行。

补充实现：

代码中状态迁移集中在 src/workflow/status.py，它定义允许迁移集合，并阻止终态继续变更。等待态和决策对象会写入 EventLog，并通过 Snapshot 保存当前上下文。这样做的意义是：即使系统中断，也能恢复到“等待人工确认”的状态，而不是自动跳过决策。

切页句：有了架构和状态机，下一步是核心机制：失败后如何选择继续、修补、重规划或止损。

第 9 页：为什么需要 CEBRA-WP

屏幕内容：阶段划分能说明流程，但不能回答失败后的运行时选择；四类动作 continue、patch_local、suffix_replan、stop；底部定义 CEBRA-WP 的作用。

建议口播：

普通阶段划分可以告诉系统“候选生成之后要做结构预测，结构预测之后要评分”，但它不能回答运行时更细的问题：当工具失败、预算升高、证据不足时，下一步应该继续、局部修补、后缀重规划，还是止损。CEBRA-WP 的作用就是在工具链执行过程中，根据约束、证据、成本和失败信息动态选择下一步恢复动作。

补充解释：

这里要特别说明，CEBRA-WP 解决的是运行时决策问题。它不负责生成新的蛋白质模型，也不替代底层结构预测工具。它处理的是“已有候选工具链在执行中如何被选择、过滤、重排和恢复”。

切页句：这套机制具体怎么做，可以概括为四步。

第 10 页：CEBRA-WP 怎么做

屏幕内容：四步主流程：候选生成、硬可行性过滤、状态更新与重排、恢复动作选择；下方三条差异：先过滤再排序、只重排合法候选、动作进入审计流程。

建议口播：

CEBRA-WP 的主流程可以压缩成四步。第一步，生成 Plan、Patch 或 Replan 的 Top-K 候选。第二步，做硬可行性过滤，检查工具是否存在、schema 是否匹配、跨步骤 I/O 是否闭合、是否违反 safety 或 budget。第三步，根据 StepResult、失败历史和 Lite state 更新运行时状态，并对候选进行重排。第四步，选择 continue、patch_local、suffix_replan 或 stop，并进入 FSM/HITL 审计流程。

关键解释：

这一页最重要的是“先硬过滤，再运行时重排”。缺工具、schema 错误、I/O 不闭合、安全违规和预算硬约束违规的候选，不进入执行排序。运行时状态只调整已经合法候选的优先级，不覆盖硬约束。

如果老师追问普通 planner 的差异：

普通 planner 更像一次性生成流程。CEBRA-WP 在候选生成之外加入硬约束过滤、运行时状态、动作选择和审计入口，因此它可以处理执行中的失败和恢复，而不是只给出初始计划。

切页句：算法机制是否真正起作用，需要看运行时状态和消融结果。

第 11 页：CEBRA-WP 的效果如何体现

屏幕内容：三个数字：21/21 lite 组 RuntimeState, 20 次 dynamic/lite 高代价调用, 28 次 fixed 高代价调用；柱状图 fixed 28、dynamic 20、lite 20；结论是状态可观测、调用更克制、结论边界清楚。

建议口播：

这一页把轻量状态和 84-run 结果连起来。第一，lite_belief_state 组 21/21 runs 产生 RuntimeState，说明算法的恢复依据能够落到日志和快照中。第二，dynamic 和 lite 的高代价调用都是 20 次，而 fixed 是 28 次，说明动态机制能减少一部分无效高代价调用。第三，成功率没有形成明显优势，所以本文不把成功率提升作为主要结论，而是把结论限定在机制可观测、恢复解释和成本控制。

补充解释：

这里可以用一句更直白的话回答老师：“CEBRA-WP 在这组实验中没有证明让成功率显著提升，但证明了运行时状态可记录、恢复决策可解释，并且相对 fixed 门控减少了 28.6% 高代价调用。”

切页句：算法讲完之后，下面看它如何落到代码结构中。

第 12 页：从设计到代码的模块落点

屏幕内容：src/api/、src/models/、src/agents/、src/workflow/、src/adapters/、src/kg/ + src/storage/，底部说明前端只展示后端状态与决策入口。

建议口播：

工程实现按模块拆分。src/api/ 提供任务、HITL、事件和报告接口；src/models/ 定义 Pydantic 契约和状态枚举；src/agents/ 实现 Planner、Executor、Safety、Summarizer；src/workflow/ 实现 FSM、PlanRunner、StepRunner 和 RuntimeEvaluator；src/adapters/ 封装具体工具；src/kg/ 和 src/storage/ 保存工具知识图谱、事件日志和快照。

补充实现：

前端不自行推导状态机，只读取后端任务状态、PendingAction 和事件链。这样可以避免前端展示与后端状态不一致。API 中与这页相关的接口包括 /tasks、/tasks/{task_id}、/tasks/{task_id}/events、/tasks/{task_id}/report、/pending-actions 和 /pending-actions/{id}/decision。

切页句：模块能对应上，还需要说明运行时失败如何被系统接住。

第 13 页：可恢复执行如何落地

屏幕内容：WorkflowContext、StepRunner、ToolAdapter / KG 三个实现对象；底部说明底层工具异常统一进入恢复状态和审计事件。

建议口播：

运行时的核心对象是 WorkflowContext，它保存任务、计划、步骤结果、安全事件、RuntimeState 和 PendingAction。StepRunner 负责输入解析、ToolAdapter 调用、有界重试、输出校验和错误归一化。ToolAdapter 统一本地和远程工具调用，ProteinToolKG 保存工具能力、I/O、成本和安全等级。

补充实现：

WorkflowContext.add_step_result 会在使用 lite belief-state 的策略下触发 runtime_state 更新。BaseToolAdapter 定义了 resolve_inputs、run_local、run_remote、healthcheck、normalize_error 等统一接口。这样各类工具的失败不会散落成普通异常，而是会转成 StepResult、RuntimeFailureContext、PendingAction 或 EventLog，进入可恢复流程。

可强调的工程难点：

落地难点在于把失败、重试、修补、重规划、人工确认、日志和快照组织成一条一致的生命周期，而不只是写一个工具调用脚本。

切页句：有实现之后，需要用测试说明系统行为是可信的。

第 14 页：13 个测试用例覆盖关键路径

屏幕内容：五类测试：API / 任务录入，HITL / FSM / 快照，Web / CLI，端到端 / 工具链，异常 / 安全 / 恢复；结果为 13 个测试用例、12 个通过、1 个 CLI 部分通过。

建议口播：

系统测试设计了 13 个用例，覆盖环境与 API、任务录入、计划候选、HITL 决策、FSM 状态迁移、快照恢复、Web 和 CLI、端到端工具链、安全边界和失败恢复。结果是 12 个通过，1 个 CLI 相关用例部分通过。部分通过的原因是 CLI 的 schema 和 task show 可用，但 timeline/report 子命令在当前版本中还不完整，所以我把它作为限制说明。

补充证据：

验证证据包括 8 个 API JSON、4 组 pytest 日志、18 张前端截图，以及 EventLog 和 Snapshot 样本。这里的测试重点放在状态迁移、候选绑定、终态不可变、等待态恢复、安全阻断和证据链一致性上。

切页句：系统测试说明工程主路径可信，下一页用消融实验说明 CEBRA-WP 机制的效果。

第 15 页：84-run 四组消融实验

屏幕内容：81/84 DONE, 21/21 lite 产生 RuntimeState, dynamic/lite 相比 fixed 节省 28.6% 高代价调用；高代价调用总数 static 21、fixed 28、dynamic 20、lite 20；结论是成功率优势不作为主要结论。

建议口播：

消融实验采用四组策略：static_top1、fixed_threshold_gate、dynamic_no_belief_state 和 lite_belief_state。总共 84 runs，每组 21 runs，覆盖 12 个 task_keys。整体结果是 81/84 进入 DONE 终态。lite_belief_state 组 21/21 runs 产生 RuntimeState，说明轻量状态可观测。高代价调用方面，fixed 是 28 次，dynamic 和 lite 都是 20 次，相比 fixed 节省 28.6%。

结论口径：

这组实验中 static 成功率最高，所以我不把 CEBRA-WP 说成成功率提升机制。本文更稳妥的结论是：CEBRA-WP 机制已实现并可执行，运行时状态可观测，恢复决策可以进入审计链，并且在高代价调用控制上相对 fixed 门控有优势。

可能被追问的数据补充：

3 个 FAILED run 分别是 fixed_threshold_gate 的 t2_ubiquitin r02、lite_belief_state 的 t2_ubiquitin r01，以及 dynamic_no_belief_state 的 t3_gbl r01。失败样本没有被隐藏，而是作为第 17-18 页备份材料，用来说明机制边界。

切页句：最后总结本文贡献、局限和后续工作。

第 16 页：贡献、局限与展望

屏幕内容：四项贡献：可恢复可审计原型、CEBRA-WP 机制、FSM/HITL 约束、证据链验证；局限：实验矩阵有限、真实后缀重规划触发不足、持久化/KG/远程服务探活待完善；结论边界限定在工作流层。

建议口播：

本文的贡献可以概括为四点。第一，实现了一个可恢复、可审计的多 Agent 蛋白质设计工作流原型，打通任务接入、候选计划、工具执行、人工决策和报告输出。第二，提出并实现 CEBRA-WP，把约束、证据、轻量状态和恢复动作纳入同一个规划闭环。第三，用 FSM、HITL、Snapshot 和 EventLog 约束 Agent 行为。第四，用系统测试和 84-run 消融实验支撑主要结论。

局限和结论边界：

当前实验矩阵仍然有限，每组 21 runs 主要支撑机制分析；真实后缀重规划在批量实验中触发不足；工程底座还可以继续完善数据库持久化、KG 动态更新和远程服务探活。本文结论限定在工作流层：系统验证了规划、执行、恢复和审计机制，候选蛋白的生物学有效性需要后续实验验证。

结束转问答：

我的汇报到这里结束，欢迎各位老师批评指正。

三、备份页使用方式

第 17 页：备份：失败样本概览

用途：回答“失败样本是否说明算法无效”“失败是否可解释”。

回答口径：

这页展示三个失败样本的类型。fixed_threshold_gate 出现 auto decision loop exhausted，说明固定门控能触发 patch，但可能进入循环耗尽；lite_belief_state 产生 RuntimeState，但单靠状态记录不一定能打破 WAITING_PATCH 循环；dynamic_no_belief_state 触发 CANDIDATE_IO_CLOSURE_BROKEN，说明硬可行性校验在执行前拦截了 I/O 不闭合候选。这些失败样本的价值在于说明系统有边界，同时失败原因和决策链可以追溯。

第 18 页：备份：失败样本如何解释

用途：需要具体解释 3 个 FAILED run 时使用。

回答口径：

fixed_threshold_gate 的 t2_ubiquitin r02 是自动决策循环耗尽；lite_belief_state 的 t2_ubiquitin r01 产生了 RuntimeState，但仍没有打破 WAITING_PATCH 循环；dynamic_no_belief_state 的

t3_gb1 r01 在执行前被 I/O 闭包校验阻断。这三个失败都保留 event log 和 snapshot, 可以说明失败没有变成不可追踪异常。

第 19 页: 备份: 测试证据索引

用途: 回答 “测试是否充分” “证据在哪里”。

回答口径:

证据按类型归档: EVD-API 是接口响应, EVD-TEST 是 pytest 日志, FIG-SV 是前端截图, EVD-LOG 是 EventLog、Snapshot、Report 样本, EVD-EXP 是实验矩阵聚合产物。系统测试覆盖 API 合约、FSM/HITL、快照恢复、安全阻断、工具链 I/O 和失败恢复, 而不只看最终 DONE。

第 20 页: 备份: 系统界面与接口证据

用途: 回答 “系统是否真的可用” “前端展示是否只是静态页面”。

回答口径:

Dashboard、Task Builder、Pending Review 和 Timeline 对应同一套后端任务、事件和决策对象。前端只展示后端状态和决策入口, 不在页面层自行推导状态机。因此同一个 task_id 可以通过 API、Web 页面和事件日志互相印证。

第 21 页: 备份: 后续工作

用途: 回答 “下一步如何改进”。

回答口径:

后续工作主要有四类: 扩大任务矩阵, 引入更多工具不可用、预算冲突、schema 错误和 I/O 闭包错误任务; 强化恢复压力, 让 patch_local、suffix_replan、terminal_stop 和 safety block 在批量矩阵中都更充分可观察; 完善数据库持久化、ProteinToolKG 动态更新、远程服务探活和配额管理; 引入更多 Agent 方法和蛋白质设计前沿方法作为比较对象。

第 22 页：谢谢

用途：汇报结束或问答结束收尾。

回答口径：

谢谢各位老师。

四、具体实现速查

1. API 与前端

主要落点：

- `src/api/main.py`: FastAPI 入口, 提供健康检查、能力 readiness、任务、事件、报告、pending action 和 decision 接口。
- `src/api/pending_views.py`: 把 PendingAction、候选、运行时状态、action utility 和证据引用整理成前端可展示结构。
- `src/api/frontend/src/pages/`: Dashboard、TaskBuilder、TaskDetail、EventTimeline 等页面。

答辩表达：

API 层的职责是把任务、事件、报告和人工决策暴露出来。前端不自行决定状态迁移, 只读取任务状态和 PendingAction, 并提交 Decision。这样可以保证 Web、CLI、API 和日志面对的是同一套任务对象。

2. 数据契约

主要落点：

- `src/models/contracts.py`: Plan、PlanStep、PlanCandidate、PatchCandidate、ReplanCandidate、StepResult、PendingAction、Decision、RuntimeState 等核心对象。
- `src/models/db.py`: InternalStatus、ExternalStatus、TaskRecord 等状态与持久化记录。
- `src/models/runtime_schemas.py`: ActionUtility、RuntimeStateSchema 等运行时解释对象。

答辩表达：

系统把 Agent 输出收束为结构化对象。Planner 输出候选，Executor 输出 StepResult，Safety 输出 SafetyResult，人工输入 Decision，系统用 Snapshot 和 EventLog 保存过程。结构化契约是系统可测试、可恢复、可审计的基础。

3. FSM / HITL

主要落点：

- `src/workflow/status.py`: 定义合法状态迁移，阻止非法跳转和终态变更。
- `src/workflow/pending_action.py`: 构造待确认动作。
- `src/workflow/decision_apply.py`: 应用人工或自动 Decision。
- `src/workflow/snapshots.py` 和 `src/storage/snapshot_store.py`: 保存和恢复任务快照。

答辩表达：

HITL 在本文中对对应状态机中的等待态。进入 `WAITING_PLAN`、`WAITING_PATCH` 或 `WAITING_REPLAN` 后，系统必须等待 Decision；Decision 应用后写入 EventLog，并根据状态机继续执行或终止。

4. CEBRA-WP 与运行时状态

主要落点：

- `src/workflow/runtime_evaluator.py`: 定义四组 runtime policy、action utility、runtime delta 和动作选择。
- `src/workflow/belief_state.py`: 用确定性规则更新 Lite belief-state。
- `src/workflow/action_features.py`: 抽取动作特征。
- `src/workflow/runtime_policy.py`: 判断当前策略是否启用 belief-state。

答辩表达：

CEBRA-WP 的实现分两部分：候选层面先进行硬可行性过滤，运行时层面再根据 StepResult、安全事件、失败上下文和预算压力更新 Lite belief-state。RuntimeEvaluator 根据状态和候选特征选择 `continue`、`patch_local`、`suffix_replan` 或 `stop`。

Lite belief-state 变量解释:

- `p_success`: 继续执行后完成任务的估计概率。
- `p_structural_failure`: 当前链路出现结构性失败的估计概率。
- `recovery_margin`: 保留有效前缀并继续恢复的余量。
- `expected_remaining_cost`: 从当前状态到任务结束的剩余成本暴露。
- `evidence_sufficiency`: 证据是否足以支持进入高代价步骤。

5. ToolAdapter 与 ProteinToolKG

主要落点:

- `src/adapters/base_tool_adapter.py`: 统一工具适配器抽象。
- `src/adapters/registry.py`: 工具注册。
- `src/adapters/*_adapter.py`: ESMFold、OpenFold、ProteinMPNN、DSSP、Foldseek、BLASTP 等具体适配器。
- `src/kg/protein_tool_kg.json` 和 `src/kg/kg_client.py`: 工具能力、I/O、成本、安全等级。

答辩表达:

ToolAdapter 把具体工具差异收束到统一接口: 解析输入、执行本地或远程调用、返回 outputs 和 metrics、健康检查、错误归一化、成本和延迟估计。ProteinToolKG 则提供工具能力和 I/O 约束, 用于候选生成和硬可行性过滤。

6. 日志、快照和证据链

主要落点:

- `src/storage/log_store.py`: EventLog JSONL。
- `src/storage/snapshot_store.py`: TaskSnapshot。
- `src/storage/filestore.py`: 产物文件管理。
- `docs/system-validation/`: 测试证据索引、API JSON、截图、pytest 日志、EventLog/Snapshot 样本。

答辩表达:

系统把状态迁移、等待态进入、Decision 应用、步骤完成、恢复动作、失败原因等写入事件链。Snapshot 保存计划、已完成步骤、pending_action 和 runtime_state。这样可以回答“系统当时为什么这么做”“失败是否可追溯”“恢复是否越过人工确认”等问题。

五、常见问题与回答

Q1: 你的系统和 AlphaFold / ESMFold 有什么关系?

回答:

AlphaFold、ESMFold 这类工具负责结构预测，属于底层蛋白质设计工具。本文工作位于它们之上，解决的是 workflow 控制问题：什么时候调用、调用前是否满足 I/O 和预算约束、失败后是否修补或重规划、关键决策如何人工确认、执行证据如何记录。因此本文不替代这些工具，也不训练新的结构预测模型。

Q2: 为什么需要多 Agent，一个 Agent 直接调用工具不行吗?

回答:

单 Agent 可以做简单 demo，但职责边界不清，容易出现规划、执行、风险判断和结论汇总混在一起的问题。本文把 Planner、Executor、Safety、Summarizer 分开，并用 Plan、StepResult、PendingAction、Decision、Snapshot 等对象连接。这样可以分别测试计划生成、执行、风险检查、人工确认和报告汇总，也能避免某个 Agent 越过状态机直接继续执行。

Q3: CEBRA-WP 和普通 planner 的区别是什么?

回答:

普通 planner 更侧重生成初始计划。CEBRA-WP 关注运行时 workflow 控制：先生成候选，再用 tool、schema、I/O、safety、budget 做硬可行性过滤；执行过程中根据 StepResult、失败历史、预算压力和 Lite belief-state 重排合法候选；最后输出 continue、patch_local、suffix_replan 或 stop，并进入 FSM/HITL 审计流程。

Q4: 为什么强调“硬约束先过滤，运行时只重排合法候选”?

回答:

因为运行时状态不应该掩盖不可执行问题。如果工具不存在、schema 不匹配、上下游 I/O 不闭合、违反 safety block 或突破硬预算，这类候选就不应该进入执行排序。Lite belief-state 只影响合法候选的优先级，不能把非法候选变成可执行候选。这是系统可解释和可审计的基础。

Q5: Lite belief-state 是完整 POMDP 吗？

回答：

可以回答：这里使用的是工程化的轻量运行时状态表示，没有采用完整 POMDP。它的目标是可解释、可持久化、可在实验中观察。它记录 p_success、p_structural_failure、recovery_margin、expected_remaining_cost、evidence_sufficiency 等变量，并用确定性更新规则响应 StepResult、安全事件和失败上下文。它服务于运行时重排和恢复动作选择，不追求完整理论建模。

Q6: 成功率没有提升，为什么还能说机制有效？

回答：

本文没有把成功率提升作为主要结论。84-run 中 static_top1 成功率最高，因此结论需要收敛到机制层。CEBRA-WP 的证据是：lite 组 21/21 runs 产生 RuntimeState，说明机制可观测；dynamic/lite 高代价调用为 20 次，fixed 为 28 次，说明动态机制能减少无效高代价调用；失败样本保留 EventLog 和 Snapshot，说明恢复和失败边界可审计。

Q7: 84-run 实验怎么设计？

回答：

实验采用四组策略，每组 21 runs，总计 84 runs。四组是 static_top1、fixed_threshold_gate、dynamic_no_belief_state 和 lite_belief_state，覆盖 12 个 task_keys。结果是 81/84 DONE，3 个 FAILED。关键指标包括成功率、first-pass success、高代价调用次数、RuntimeState 可观测率、patch/replan 事件和执行时间。

Q8: 为什么 fixed 的高代价调用更多？

回答：

`fixed_threshold_gate` 使用静态门控, 运行时更容易在发现问题后触发 `patch`。实验中 `fixed` 产生 28 次高代价调用, 其中一部分来自 `patch` 后重跑结构相关步骤。`dynamic` 和 `lite` 通过运行时观测或 `belief-state` 重排, 减少了进入这类高代价修补路径的情况, 因此高代价调用都是 20 次。

Q9: 第 14 页的 CLI 部分通过会不会影响系统结论?

回答:

不会影响主结论, 但需要如实说明。CLI 的 `intake schema` 和 `task show` 已可用, `timeline/report` 子命令在当前版本中还不完整, 所以 TC-S08 记为部分通过。系统的主验证来自 API、Web、FSM/HITL、快照恢复、安全阻断、端到端流程和消融实验, 这些路径的证据是完整的。

Q10: HITL 如何保证可审计?

回答:

HITL 由 `PendingAction`、`Decision`、`EventLog` 和 `Snapshot` 共同约束。进入等待态时系统生成 `PendingAction` 和候选; 用户或自动策略提交 `Decision`; `Decision` 应用后写入事件链; `Snapshot` 保存当时的计划、已完成步骤、`pending_action` 和 `runtime_state`。因此事后可以追溯谁在什么状态下接受了哪个候选, 以及系统之后如何继续执行。

Q11: 失败样本说明什么?

回答:

失败样本主要说明机制边界。`fixed_threshold_gate` 的失败说明固定门控可能陷入 `patch` 循环; `lite_belief_state` 的失败说明有 `RuntimeState` 仍不保证每次打破循环; `dynamic_no_belief_state` 的失败说明 I/O 闭包硬校验会在执行前阻断不可执行候选。这些失败没有被包装成成功, 而是通过 `EventLog` 和 `Snapshot` 保留下来, 用于说明边界和可追溯性。

Q12: 你的个人工作主要体现在哪里?

回答:

个人工作主要在四个方面。第一，完成 workflow 控制层的系统架构设计，包括五层架构、多 Agent 职责和 FSM/HITL。第二，提出并实现 CEBRA-WP 机制，包括候选生成、硬过滤、运行时状态、恢复动作和审计流程。第三，完成工程实现，包括 API、模型契约、Workflow、ToolAdapter、ProteinToolKG、日志快照和 Web 工作台。第四，完成系统测试与实验验证，包括 13 个系统测试用例和 84-run 四组消融实验。

Q13: 系统当前最大的不足是什么？

回答：

主要不足有三点。第一，实验矩阵规模有限，每组 21 runs 主要支撑机制分析。第二，真实后缀重规划在批量实验中触发不足，suffix_replan 的压力测试还需要增强。第三，工程底座仍有原型属性，数据库持久化、ProteinToolKG 动态更新、远程服务探活和配额管理可以继续完善。

Q14: 如果继续做，最优先改进什么？

回答：

我会优先扩大任务矩阵和恢复压力场景，让工具不可用、预算冲突、schema 错误、I/O 闭包错误、安全阻断、patch_local、suffix_replan 和 terminal_stop 都在批量实验中充分出现。其次完善持久化和远程服务探活，让系统从原型进一步接近长期运行的科研工作流平台。

六、最后记忆点

答辩时可以把 CEBRA-WP 压缩成三句话：

1. CEBRA-WP 面向运行时恢复决策，底层蛋白质生成不在本文范围内。
2. 它先用 tool、schema、I/O、safety、budget 做硬过滤，再用运行时状态重排合法候选。
3. 它输出 continue、patch_local、suffix_replan、stop 四类动作，并通过 FSM/HITL 进入可审计流程。

整篇答辩的收束句：

本文在工作流层验证了一个可规划、可执行、可恢复、可审计的多 Agent 蛋白质设计系统原型；实验结果支撑机制可观测和成本控制，但不扩大为蛋白质生物学有效性的结论。